ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Создание приложения интернета вещей и веб-интерфейса

по курсу: ИНТЕРНЕТ ВЕЩЕЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		Д.Ю. Бычков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков создания и запуска приложения интернета вещей.

2 Задание

Вариант задания: Умный дом.

Создать приложение на Flask. Реализовать классы и их иерархию, согласно сданной первой лабораторной работе, показать работу методов этих классов. (Не нужно писать предполагаемый метод целиком, на данном этапе достаточно вывести в лог сервера сообщение о том, что метод был запущен).

3 Результат проделанной работы

В ходе работы были созданы соответствующие классы и методы. На рисунке 1 продемонстрирован лог успешно выполненных методов класса. На рисунке 2 продемонстрировано информирование на сайте о успешном выполнении функционала. На рисунке 3 изображена реализация класса умных штор.

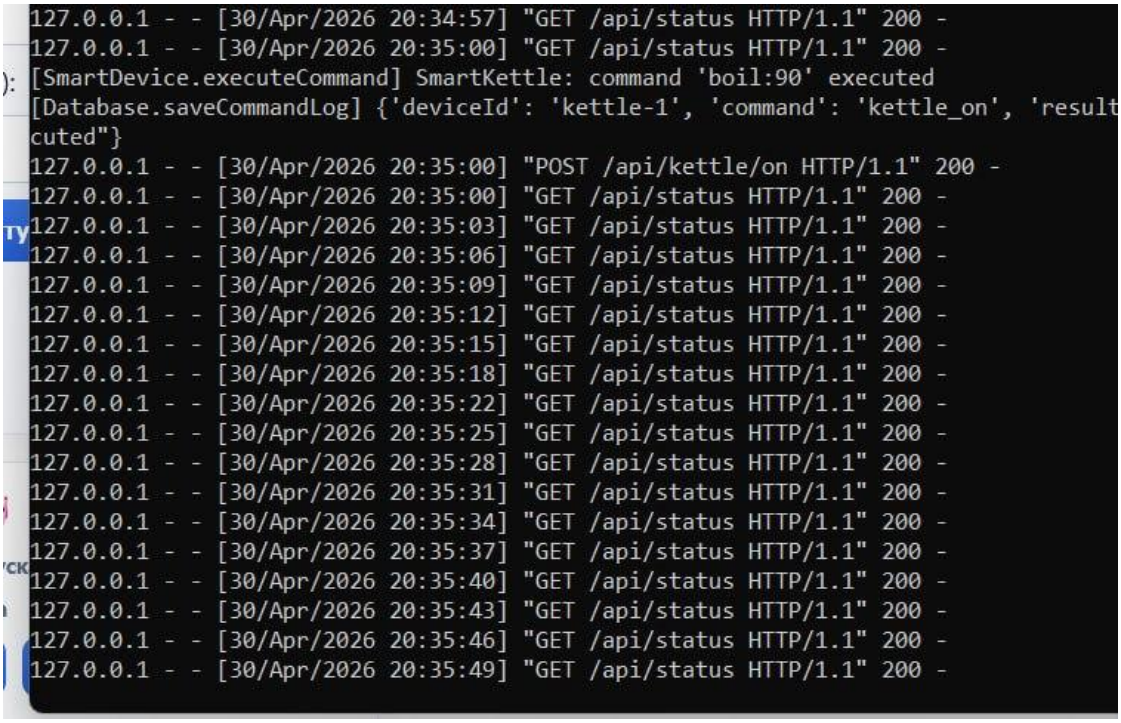

Рисунок 1 — Лог выполнения методов

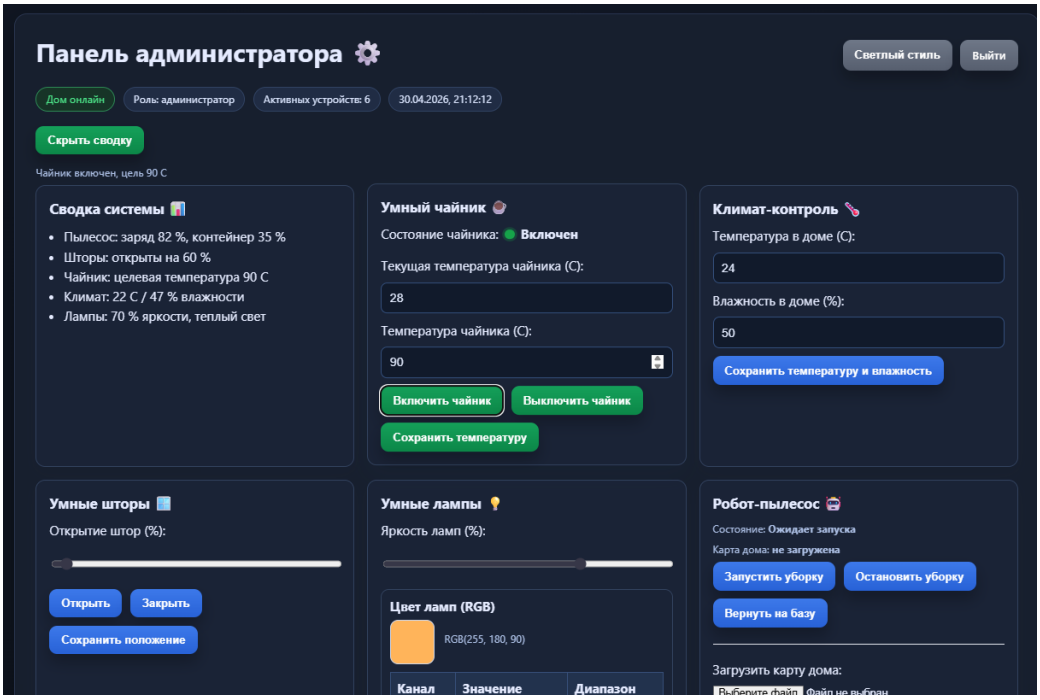

Рисунок 2 — Включение чайника на веб-сервере

```python
class SmartCurtains(SmartDevice):
    def __init__(self, device_id: str, name: str):
        super().__init__(device_id, name)
        self.positionPercent: int = 0
        self.mode: str = "manual"

    def connect(self) -> dict:
        self.status = "online"
        payload = {"id": self.id, "positionPercent": self.positionPercent, "mode": self.mode}
        print(f"[SmartCurtains.connect] {payload}")
        return payload

    def open(self, percent: int) -> str:
        self.positionPercent = max(0, min(100, percent))
        msg = f"{self.name}: opened to {self.positionPercent}%"
        print(f"[SmartCurtains.open] {msg}")
        return msg

    def close(self) -> str:
        self.positionPercent = 0
        msg = f"{self.name}: closed"
        print(f"[SmartCurtains.close] {msg}")
        return msg
```

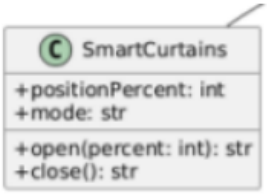

Рисунок 3 — Пример реализации класса из прошлой лабораторной работы

4 Вывод

В ходе выполнения лабораторной работы были приобретены навыки создания и запуска приложения интернета вещей.

ПРИЛОЖЕНИЕ А

Листинг кода файла things.py:

```
import abc
from typing import Any



classef __init__( SmartDevice(abc.ABC):
    dself, device_id: str, name: str):
        self.id: str = device_id
        self.name: str = name
        self.status: str = "offline"
        self.schedule: list = []
        print(f"[SmartDevice.__init__] created {self.name} ({self.id})")

    @abc.abstractmethod
    def connect(self) -> dict:
        pass

    def executeCommand(self, command: str) -> str:
        self.status = f"executed: {command}"
        msg = f"{self.name}: command '{command}' executed"
        print(f"[SmartDevice.executeCommand] {msg}")
        return msg

    def saveSchedule(self, scheduleData: list) -> str:
        self.schedule = list(scheduleData)
        msg = f"{self.name}: schedule saved ({len(self.schedule)} items)"
        print(f"[SmartDevice.saveSchedule] {msg}")
        return msg

    def updateSchedule(self, scheduleData: list) -> str:
        self.schedule = list(scheduleData)
        msg = f"{self.name}: schedule updated ({len(self.schedule)} items)"
        print(f"[SmartDevice.updateSchedule] {msg}")
        return msg
```

```
class RobotVacuum(SmartDevice):
    def __init__(self, device_id: str, name: str):
        super().__init__(device_id, name)
        self.batteryLevel: int = 85
        self.dustContainerLevel: int = 20
        self.roomMapVersion: int = 1
        self.roomMap: list = ["hall", "kitchen", "bedroom"]

    def connect(self) -> dict:
        self.status = "online"
        payload = {"id": self.id, "status": self.status, "batteryLevel": self.batteryLevel}
        print(f"[RobotVacuum.connect] {payload}")
        return payload

    def startCleaning(self, mode: str, roomMap: list) -> str:
        self.status = f"cleaning ({mode})"
        msg = f"{self.name}: cleaning started, mode={mode}, rooms={roomMap}"
        print(f"[RobotVacuum.startCleaning] {msg}")
        return msg

    def updateRoomMap(self, newRoomMap: list) -> str:
        self.roomMapVersion += 1
        self.roomMap = list(newRoomMap)
        msg = f"{self.name}: room map updated to v{self.roomMapVersion}"
        print(f"[RobotVacuum.updateRoomMap] {msg}")
        return msg

    def goToDock(self) -> str:
        self.status = "docked"
        msg = f"{self.name}: returned to dock"
        print(f"[RobotVacuum.goToDock] {msg}")
        return msg
```

```
class SmartCurtains(SmartDevice):
    def __init__(self, device_id: str, name: str):
        super().__init__(device_id, name)
        self.positionPercent: int = 0
        self.mode: str = "manual"

    def connect(self) -> dict:
        self.status = "online"
        payload = {"id": self.id, "positionPercent": self.positionPercent, "mode": self.mode}
        print(f"[SmartCurtains.connect] {payload}")
        return payload

    def open(self, percent: int) -> str:
        self.positionPercent = max(0, min(100, percent))
        msg = f"{self.name}: opened to {self.positionPercent}%"
        print(f"[SmartCurtains.open] {msg}")
        return msg

    def close(self) -> str:
        self.positionPercent = 0
        msg = f"{self.name}: closed"
        print(f"[SmartCurtains.close] {msg}")
        return msg



class SmartKettle(SmartDevice):
    def __init__(self, device_id: str, name: str):
        super().__init__(device_id, name)
        self.waterLevelMl: int = 1200
        self.targetTemperature: int = 90
        self.currentWaterTemperature: float = 25.0
        self.isBoiling: bool = False

    def connect(self) -> dict:
```

```
            self.status = "online"
            payload = {
                "id": self.id,
                "waterLevelMl": self.waterLevelMl,
                "targetTemperature": self.targetTemperature,
                "currentWaterTemperature": self.currentWaterTemperature,
                "isBoiling": self.isBoiling,
            }
            print(f"[SmartKettle.connect] {payload}")
            return payload

        def boil(self, targetTemp: int) -> str:
            self.targetTemperature = targetTemp
            self.currentWaterTemperature = float(targetTemp)
            self.isBoiling = True
            msg = f"{self.name}: boiled to {self.targetTemperature} C"
            print(f"[SmartKettle.boil] {msg}")
            self.isBoiling = False
            return msg

        def keepWarm(self, minutes: int) -> str:
            msg = f"{self.name}: keep warm for {minutes} minutes"
            print(f"[SmartKettle.keepWarm] {msg}")
            return msg

        def getCurrentWaterTemperature(self) -> float:
            print(f"[SmartKettle.getCurrentWaterTemperature]
{self.currentWaterTemperature}")
            return self.currentWaterTemperature


    class TemperatureControl(SmartDevice):
        def __init__(self, device_id: str, name: str):
            super().__init__(device_id, name)
            self.temperature: float = 22.0
```

```
        self.targetTemperature: float = 24.0

    def connect(self) -> dict:
        self.status = "online"
        payload = {"id": self.id, "temperature": self.temperature, "targetTemperature": self.targetTemperature}
        print(f"[TemperatureControl.connect] {payload}")
        return payload

    def setTargetTemperature(self, temp: float) -> str:
        self.targetTemperature = float(temp)
        msg = f"{self.name}: target temperature set to {self.targetTemperature} C"
        print(f"[TemperatureControl.setTargetTemperature] {msg}")
        return msg

    def getCurrentTemperature(self) -> float:
        print(f"[TemperatureControl.getCurrentTemperature] {self.temperature}")
        return self.temperature


class HumidityControl(SmartDevice):
    def __init__(self, device_id: str, name: str):
        super().__init__(device_id, name)
        self.humidity: float = 45.0
        self.targetHumidity: float = 50.0

    def connect(self) -> dict:
        self.status = "online"
        payload = {"id": self.id, "humidity": self.humidity, "targetHumidity": self.targetHumidity}
        print(f"[HumidityControl.connect] {payload}")
        return payload

    def setTargetHumidity(self, humidity: float) -> str:
        self.targetHumidity = float(humidity)
```

```
        msg = f"{self.name}: target humidity set to {self.targetHumidity} %"
        print(f"[HumidityControl.setTargetHumidity] {msg}")
        return msg

    def getCurrentHumidity(self) -> float:
        print(f"[HumidityControl.getCurrentHumidity] {self.humidity}")
        return self.humidity


class SmartLighting(SmartDevice):
    def __init__(self, device_id: str, name: str):
        super().__init__(device_id, name)
        self.brightness: int = 70
        self.colorTemperature: int = 3500

    def connect(self) -> dict:
        self.status = "online"
        payload = {
            "id": self.id,
            "brightness": self.brightness,
            "colorTemperature": self.colorTemperature,
        }
        print(f"[SmartLighting.connect] {payload}")
        return payload

    def setBrightness(self, value: int) -> str:
        self.brightness = max(0, min(100, value))
        msg = f"{self.name}: brightness set to {self.brightness}%"
        print(f"[SmartLighting.setBrightness] {msg}")
        return msg

    def setColorTemperature(self, value: int) -> str:
        self.colorTemperature = max(2000, min(6500, value))
        msg = f"{self.name}: color temperature set to {self.colorTemperature}K"
        print(f"[SmartLighting.setColorTemperature] {msg}")
```

```
        return msg


class Database:
    def __init__(self, name: str):
        self.name: str = name
        self.deviceScheduleList: dict[str, list] = {}
        self._deviceData: dict[str, list] = {}
        self._commandLog: list[dict[str, str]] = []
        print(f"[Database.__init__] db '{self.name}' initialized")

    def saveData(self, deviceId: str, payload: dict) -> None:
        self._deviceData.setdefault(deviceId, []).append(payload)
        print(f"[Database.saveData] {deviceId}: {payload}")

    def saveCommandLog(self, deviceId: str, command: str, result: str) -> None:
        entry = {"deviceId": deviceId, "command": command, "result": result}
        self._commandLog.append(entry)
        print(f"[Database.saveCommandLog] {entry}")

    def saveDeviceSchedule(self, deviceId: str, scheduleData: list) -> None:
        self.deviceScheduleList[deviceId] = list(scheduleData)
        print(f"[Database.saveDeviceSchedule] {deviceId}: {scheduleData}")

    def getDeviceHistory(self, deviceId: str) -> list:
        history = self._deviceData.get(deviceId, [])
        print(f"[Database.getDeviceHistory] {deviceId}: {len(history)} records")
        return history


class MainControlUnit:
    def __init__(self, database: Database):
        self.deviceList: list[SmartDevice] = []
        self.command_queue: list[dict] = []
        self._database = database
```

```
        print("[MainControlUnit.__init__] main control unit created")

    def addDevice(self, device: SmartDevice) -> str:
        self.deviceList.append(device)
        msg = f"Device '{device.name}' added"
        print(f"[MainControlUnit.addDevice] {msg}")
        return msg

    def dltDevice(self, deviceId: str) -> str:
        before = len(self.deviceList)
        self.deviceList = [d for d in self.deviceList if d.id != deviceId]
        after = len(self.deviceList)
        msg = "Device deleted" if after < before else "Device not found"
        print(f"[MainControlUnit.dltDevice] {deviceId}: {msg}")
        return msg

    def createCommand(self, deviceId: str, command: str) -> dict:
        payload = {"deviceId": deviceId, "command": command}
        self.command_queue.append(payload)
        print(f"[MainControlUnit.createCommand] {payload}")
        return payload

    def getData(self, deviceId: str) -> dict:
        device = next((d for d in self.deviceList if d.id == deviceId), None)
        if device is None:
            print(f"[MainControlUnit.getData] {deviceId}: not found")
            return {"error": "device not found"}
        data = device.connect()
        print(f"[MainControlUnit.getData] {deviceId}: {data}")
        return data

    def sentToDataBase(self, payload: dict) -> None:
        deviceId = str(payload.get("deviceId", "unknown"))
        self._database.saveData(deviceId, payload)
        print(f"[MainControlUnit.sentToDataBase] payload sent for {deviceId}")
```

```
    def dispatchCommand(self, deviceId: str, command: str) -> str:
        device = next((d for d in self.deviceList if d.id == deviceId), None)
        if device is None:
            result = "device not found"
            self._database.saveCommandLog(deviceId, command, result)
            print(f"[MainControlUnit.dispatchCommand] {deviceId}: {result}")
            return result
        result = device.executeCommand(command)
        self._database.saveCommandLog(deviceId, command, result)
        return result


class ChildUI:
    def __init__(self, mcu: MainControlUnit):
        self._mcu = mcu

    def showBasicStatus(self, data: dict) -> None:
        print(f"[ChildUI.showBasicStatus] {data}")

    def controlAllowedDevices(self) -> str:
        msg = "ChildUI: control of allowed devices enabled"
        print(f"[ChildUI.controlAllowedDevices] {msg}")
        return msg

    def sendCommand(self, deviceId: str, command: str) -> bool:
        result = self._mcu.dispatchCommand(deviceId, command)
        ok = "not found" not in result
        print(f"[ChildUI.sendCommand] device={deviceId}, ok={ok}")
        return ok


class ParentUI:
    def __init__(self, mcu: MainControlUnit):
        self._mcu = mcu
```

```
    def showFullStatus(self, data: dict) -> None:
        print(f"[ParentUI.showFullStatus] {data}")

    def configureAutomationRules(self) -> str:
        msg = "ParentUI: automation rules configured"
        print(f"[ParentUI.configureAutomationRules] {msg}")
        return msg

    def sendCommand(self, deviceId: str, command: str) -> bool:
        result = self._mcu.dispatchCommand(deviceId, command)
        ok = "not found" not in result
        print(f"[ParentUI.sendCommand] device={deviceId}, ok={ok}")
        return ok


def build_system() -> dict[str, Any]:
    db = Database("SmartHomeDB")
    mcu = MainControlUnit(db)

    devices: list[SmartDevice] = [
        RobotVacuum("vacuum-1", "RobotVacuum"),
        SmartCurtains("curtains-1", "SmartCurtains"),
        SmartKettle("kettle-1", "SmartKettle"),
        TemperatureControl("temp-1", "TemperatureControl"),
        HumidityControl("humidity-1", "HumidityControl"),
        SmartLighting("light-1", "SmartLighting"),
    ]
    for device in devices:
        mcu.addDevice(device)

    child_ui = ChildUI(mcu)
    parent_ui = ParentUI(mcu)

    return {
```

```
        "db": db,
        "mcu": mcu,
        "devices": devices,
        "child_ui": child_ui,
        "parent_ui": parent_ui,
    }
```

Файла app.py

```
from flask import Flask, jsonify, render_template, request

import things

app = Flask(__name__, template_folder="server", static_folder="server", static_url_path="")

system = things.build_system()
db = system["db"]
mcu = system["mcu"]
devices = system["devices"]
child_ui = system["child_ui"]
parent_ui = system["parent_ui"]

robot = devices[0]
curtains = devices[1]
kettle = devices[2]
temperature = devices[3]
humidity = devices[4]
lighting = devices[5]

runtime_state = {
    "kettle_on": False,
    "vacuum_state": "Ожидает запуска",
    "map_name": "",
    "lights_on": True,
    "light_rgb": {"r": 255, "g": 180, "b": 90},
```

```
    "scene_last_saved": "",
}


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/index.html")
def index_html():
    return render_template("index.html")


@app.route("/admin.html")
def admin_html():
    return render_template("admin.html")


@app.route("/user.html")
def user_html():
    return render_template("user.html")


def _make_status() -> dict:
    if runtime_state["kettle_on"]:
        kettle.currentWaterTemperature = min(float(kettle.targetTemperature),
kettle.currentWaterTemperature + 2.0)
        if kettle.currentWaterTemperature >= float(kettle.targetTemperature):
            runtime_state["kettle_on"] = False
            kettle.isBoiling = False
    else:
        kettle.currentWaterTemperature = max(24.0, kettle.currentWaterTemperature - 1.0)

    return {
```

```
        "kettle": {
            "is_on": runtime_state["kettle_on"],
            "state_text": "Включен" if runtime_state["kettle_on"] else "Выключен",
            "current_temp": kettle.currentWaterTemperature,
            "target_temp": kettle.targetTemperature,
        },
        "vacuum": {
            "state": runtime_state["vacuum_state"],
            "battery": robot.batteryLevel,
            "map_name": runtime_state["map_name"] or "не загружена",
        },
        "curtains": {"position_percent": curtains.positionPercent},
        "lights": {
            "brightness": lighting.brightness,
            "color_temp": lighting.colorTemperature,
            "rgb": runtime_state["light_rgb"],
            "is_on": runtime_state["lights_on"],
            "state_text": "Включены" if runtime_state["lights_on"] else "Выключены",
        },
        "climate": {
            "temperature": temperature.temperature,
            "target_temperature": temperature.targetTemperature,
            "humidity": humidity.humidity,
            "target_humidity": humidity.targetHumidity,
        },
        "devices": [d.name for d in mcu.deviceList],
        "scene_last_saved": runtime_state["scene_last_saved"],
    }


@app.route("/api/status")
def api_status():
    return jsonify(_make_status())
```

```
@app.route("/api")
@app.route("/api/")
def api_root():
    return jsonify({"ok": True, "message": "LR2 API is running", "status": _make_status()})


@app.route("/api/kettle/on", methods=["POST"])
def api_kettle_on():
    payload = request.get_json(silent=True) or {}
    target = int(payload.get("target_temp", kettle.targetTemperature))
    kettle.targetTemperature = max(40, min(100, target))
    kettle.isBoiling = True
    runtime_state["kettle_on"] = True
    msg = kettle.executeCommand(f"boil:{kettle.targetTemperature}")
    db.saveCommandLog(kettle.id, "kettle_on", msg)
    return jsonify({"ok": True, "message": f"Чайник включен, цель
{kettle.targetTemperature} C"})


@app.route("/api/kettle/off", methods=["POST"])
def api_kettle_off():
    kettle.isBoiling = False
    runtime_state["kettle_on"] = False
    msg = kettle.executeCommand("off")
    db.saveCommandLog(kettle.id, "kettle_off", msg)
    return jsonify({"ok": True, "message": "Чайник выключен"})


@app.route("/api/kettle/target", methods=["POST"])
def api_kettle_target():
    payload = request.get_json(silent=True) or {}
    target = int(payload.get("target_temp", kettle.targetTemperature))
    kettle.targetTemperature = max(40, min(100, target))
    msg = f"Целевая температура чайника сохранена: {kettle.targetTemperature} C"
    return jsonify({"ok": True, "message": msg, "target_temp": kettle.targetTemperature})
```

```
@app.route("/api/curtains/open", methods=["POST"])
def api_curtains_open():
    payload = request.get_json(silent=True) or {}
    percent = int(payload.get("percent", 100))
    result = curtains.open(percent)
    return jsonify({"ok": True, "message": result, "position": curtains.positionPercent})


@app.route("/api/curtains/close", methods=["POST"])
def api_curtains_close():
    result = curtains.close()
    return jsonify({"ok": True, "message": result, "position": curtains.positionPercent})


@app.route("/api/curtains/set", methods=["POST"])
def api_curtains_set():
    payload = request.get_json(silent=True) or {}
    percent = int(payload.get("percent", curtains.positionPercent))
    percent = max(0, min(100, percent))
    result = curtains.open(percent)
    return jsonify({"ok": True, "message": f"Положение штор сохранено: {percent}%",
"position": curtains.positionPercent, "result": result})


@app.route("/api/lights/apply", methods=["POST"])
def api_lights_apply():
    payload = request.get_json(silent=True) or {}
    brightness = int(payload.get("brightness", lighting.brightness))
    r = int(payload.get("r", runtime_state["light_rgb"]["r"]))
    g = int(payload.get("g", runtime_state["light_rgb"]["g"]))
    b = int(payload.get("b", runtime_state["light_rgb"]["b"]))
    lighting.setBrightness(brightness)
    runtime_state["light_rgb"] = {
```

```
            "r": max(0, min(255, r)),
            "g": max(0, min(255, g)),
            "b": max(0, min(255, b)),
        }
        avg = int((runtime_state["light_rgb"]["r"] + runtime_state["light_rgb"]["g"] + runtime_state["light_rgb"]["b"]) / 3)
        lighting.setColorTemperature(2000 + int((avg / 255) * 4500))
        return jsonify({"ok": True, "message": "Настройки ламп применены", "lights": _make_status()["lights"]})


@app.route("/api/lights/on", methods=["POST"])
def api_lights_on():
    runtime_state["lights_on"] = True
    if lighting.brightness == 0:
        lighting.setBrightness(70)
    return jsonify({"ok": True, "message": "Лампы включены", "lights": _make_status()["lights"]})


@app.route("/api/lights/off", methods=["POST"])
def api_lights_off():
    runtime_state["lights_on"] = False
    lighting.setBrightness(0)
    return jsonify({"ok": True, "message": "Лампы выключены", "lights": _make_status()["lights"]})


@app.route("/api/vacuum/start", methods=["POST"])
def api_vacuum_start():
    runtime_state["vacuum_state"] = "Запущен"
    result = robot.startCleaning("auto", robot.roomMap)
    return jsonify({"ok": True, "message": result, "state": runtime_state["vacuum_state"]})
```

```
@app.route("/api/vacuum/pause", methods=["POST"])
def api_vacuum_pause():
    runtime_state["vacuum_state"] = "Ожидает запуска"
    return jsonify({"ok": True, "message": "Уборка приостановлена", "state": runtime_state["vacuum_state"]})


@app.route("/api/vacuum/dock", methods=["POST"])
def api_vacuum_dock():
    runtime_state["vacuum_state"] = "Ожидает запуска"
    result = robot.goToDock()
    return jsonify({"ok": True, "message": result, "state": runtime_state["vacuum_state"]})


@app.route("/api/map/upload", methods=["POST"])
def api_map_upload():
    payload = request.get_json(silent=True) or {}
    map_name = str(payload.get("map_name", "")).strip()
    if not map_name:
        return jsonify({"ok": False, "message": "Имя карты не передано"}), 400
    runtime_state["map_name"] = map_name
    robot.updateRoomMap(["hall", "kitchen", "bedroom"])
    return jsonify({"ok": True, "message": f"Карта '{map_name}' загружена", "map_name": map_name})


@app.route("/api/map/view")
def api_map_view():
    map_name = runtime_state["map_name"]
    if not map_name:
        return jsonify({"ok": False, "message": "Карта дома еще не загружена"}), 404
    return jsonify({"ok": True, "message": f"Открыта карта: {map_name}", "map_name": map_name})
```

```
@app.route("/api/devices")
def api_devices():
    return jsonify({"ok": True, "devices": [d.name for d in mcu.deviceList]})


@app.route("/api/devices/add", methods=["POST"])
def api_devices_add():
    payload = request.get_json(silent=True) or {}
    name = str(payload.get("name", "")).strip()
    if not name:
        return jsonify({"ok": False, "message": "Пустое имя устройства"}), 400
    device_id = f"custom-{len(mcu.deviceList) + 1}"
    new_device = things.SmartLighting(device_id, name)
    mcu.addDevice(new_device)
    return jsonify({"ok": True, "message": f"Устройство '{name}' добавлено", "devices":
[d.name for d in mcu.deviceList]})


@app.route("/api/devices/remove", methods=["POST"])
def api_devices_remove():
    payload = request.get_json(silent=True) or {}
    index = int(payload.get("index", -1))
    if index < 0 or index >= len(mcu.deviceList):
        return jsonify({"ok": False, "message": "Некорректный индекс"}), 400
    name = mcu.deviceList[index].name
    del mcu.deviceList[index]
    return jsonify({"ok": True, "message": f"Устройство '{name}' удалено", "devices":
[d.name for d in mcu.deviceList]})


@app.route("/api/climate/apply", methods=["POST"])
def api_climate_apply():
    payload = request.get_json(silent=True) or {}
    t = float(payload.get("target_temperature", temperature.targetTemperature))
    h = float(payload.get("target_humidity", humidity.targetHumidity))
```

```
        msg_t = temperature.setTargetTemperature(t)
        msg_h = humidity.setTargetHumidity(h)
        return jsonify({"ok": True, "message": f"{msg_t}; {msg_h}", "climate": _make_status()["climate"]})


    @app.route("/api/scene/save", methods=["POST"])
    def api_scene_save():
        payload = request.get_json(silent=True) or {}
        name = str(payload.get("name", "")).strip() or "Без названия"
        scene_dt = f"{payload.get('date', '')} {payload.get('time', '')}".strip()
        runtime_state["scene_last_saved"] = f"{name} ({scene_dt})"
        return jsonify({"ok": True, "message": f"Сценарий '{name}' сохранен", "scene": runtime_state["scene_last_saved"]})


    @app.route("/demo")
    def demo():
        print("\n===== LR2 DEMO START =====")

        robot = devices[0]
        curtains = devices[1]
        kettle = devices[2]
        temperature = devices[3]
        humidity = devices[4]
        lighting = devices[5]

        results = {
            "connect": [device.connect() for device in devices],
            "robot": [
                robot.startCleaning("eco", ["hall", "kitchen"]),
                robot.updateRoomMap(["hall", "kitchen", "bedroom", "bathroom"]),
                robot.goToDock(),
            ],
            "curtains": [curtains.open(65), curtains.close()],
```

```
        "kettle": [kettle.boil(95), kettle.keepWarm(15), kettle.getCurrentWaterTemperature()],
        "temperature": [temperature.setTargetTemperature(23.5), temperature.getCurrentTemperature()],
        "humidity": [humidity.setTargetHumidity(48.0), humidity.getCurrentHumidity()],
        "lighting": [lighting.setBrightness(80), lighting.setColorTemperature(4200)],
        "schedules": [
            kettle.saveSchedule(["07:00 boil 90C"]),
            lighting.updateSchedule(["20:00 brightness 40%"]),
        ],
    }

    cmd1 = mcu.createCommand("kettle-1", "boil:95")
    cmd2 = mcu.createCommand("light-1", "setBrightness:80")
    res1 = mcu.dispatchCommand("kettle-1", "boil:95")
    res2 = mcu.dispatchCommand("light-1", "setBrightness:80")
    data_temp = mcu.getData("temp-1")
    mcu.sentToDataBase({"deviceId": "temp-1", "value": data_temp})
    db.saveDeviceSchedule("kettle-1", ["07:00 boil 90C"])
    history = db.getDeviceHistory("temp-1")

    child_ok = child_ui.sendCommand("curtains-1", "open:50")
    child_ui.controlAllowedDevices()
    child_ui.showBasicStatus(mcu.getData("curtains-1"))

    parent_ok = parent_ui.sendCommand("light-1", "setColorTemperature:4000")
    parent_ui.configureAutomationRules()
    parent_ui.showFullStatus(mcu.getData("light-1"))

    report = {
        "results": results,
        "mcu": {
            "created_commands": [cmd1, cmd2],
            "dispatch_results": [res1, res2],
            "queue_size": len(mcu.command_queue),
```

```
        },
        "database": {"history_temp_count": len(history)},
        "ui": {"child_send_ok": child_ok, "parent_send_ok": parent_ok},
    }

    print("===== LR2 DEMO END =====\n")
    return jsonify(report)


if __name__ == "__main__":
    app.run(debug=True)
```